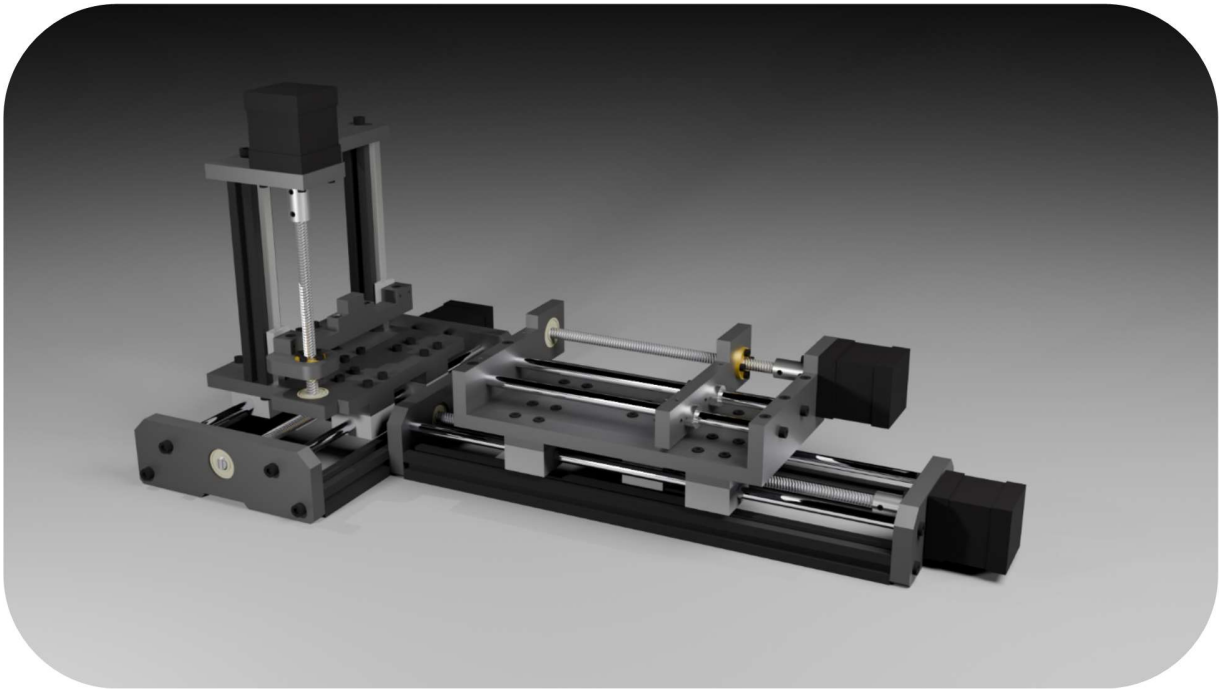
CNC MICROPHOTOGRAPHY

CONTROL SOFTWARE

Complete Technical Documentation — v2



CMCS.py — Python 3 / Tkinter / GRBL

July 2025 – May 2026

Louis Villiers

Table of Contents

Table of Contents 2
1. Overview 6
1.1 Purpose 6
1.2 Software Architecture 6
1.3 Dependencies 7
2. CNC Connection and Manual Control 8
2.1 Serial Connection 8
2.2 Manual XYZ Jogging 8
2.3 Movement Parameters 8
2.4 Thread-Safe Serial Communication 9
2.5 Real-Time Position Display 9
3. Hardware Profiles 10
3.1 Lens Profile 10
3.2 Camera Profile 10
3.3 Machine Profile 10
3.4 Deleting a Profile 11
3.5 Persistence 11
4. Optical Calculation Engine 12
4.1 Focal Length 12
4.2 Effective Magnification and Total Extension 12
4.3 Field of View (FOV) 12
4.4 Depth of Field (DoF) 12
4.5 Step Sizes 12
4.6 M Axis Optical Compensation 13
5. Acquisition Sequence 14
5.1 Defining the Scan Volume 14

5.2 Auto-Calculate and Time Estimation 14
5.3 Pre-Launch Validation 14
5.4 Scan Algorithm 14
5.5 GRBL Alarm Detection 15
5.6 Sequence Controls 15
6. CSV Session Logging 17
6.1 Write Strategy 17
6.2 File Structure 17
Metadata Header 17
Data Columns 17
Summary Footer 18
7. Magnification Panel (M Axis) 19
7.1 Real-Time Displays 19
7.2 M Step 19
7.3 M Moves with Optical Compensation 19
8. 3D Isometric Preview 21
8.1 Visual Elements 21
8.2 Draw Order (Painter's Algorithm) 21
8.3 Corrected Start / End Labels 22
8.4 Isometric Projection 22
9. GRBL Console 23
10. User Interface and Layout 24
10.1 Panel Organisation 24
10.2 Drag & Drop Layout 24
10.3 Dark Theme 24
11. Typical Workflow 25
Step 1 — Connect 25

Step 2 — Hardware Setup 25
Step 3 — Positioning 25
Step 4 — Sequence Setup 25
Step 5 — Acquisition 25
Step 6 — Data Retrieval 25
12. Complete Parameter Reference 27
12.1 Sequence Parameters 27
12.2 Connection Parameters 27
12.3 Optical and System Constants 27
12.4 config.json Key Reference 28
13. Complementary Software Suite 29
13.1 Post-Watcher (PW) 29
13.1.1 Purpose 29
13.1.2 Workflow Phases 29
13.1.3 Output Format Selection 30
13.1.4 Safety Features 31
13.2 Dust Map Profiler 31
13.2.1 Purpose 31
13.2.2 Algorithm 31
13.2.3 Usage in Post-Watcher 32
13.2.4 Design Notes 32
13.3 Poisson Profiler 32
13.3.1 Purpose 32
13.3.2 Theory 32
13.3.3 Algorithm 33
13.3.4 Visualization 33
13.3.5 Output 33

13.4 Calibration Pipeline Overview 33
13.5 Sidecar JSON Format Specification 34
13.5.1 Rationale 34
13.5.2 Schema 34
13.5.3 Example 35
13.5.4 Export Tree Structure 35
13.5.5 Future Stacking Engine Integration 36
14. Complete Software Dependencies 37

1. Overview

CNC Microphotography Control is a desktop application written in Python 3, using the Tkinter GUI library and the GRBL serial protocol to drive a CNC machine dedicated to focus-stacking microphotography.

The software orchestrates the entire acquisition pipeline: connecting to the controller board, automatically positioning the stage, triggering the camera, logging coordinates to CSV, and providing a real-time 3D preview.

1.1 Purpose

Focus stacking involves photographing a subject at successive focus positions, then blending the images in post-processing to achieve full-depth sharpness. This software fully automates the mechanical side:

- **XYZ Movements** — 3D mosaic scan of the subject
- **M Axis** — controls optical tube extension and effective magnification
- **Camera Trigger** — digital signal via GRBL I/O port
- **Optical Calculations** — step sizes and scan bounds computed automatically from hardware profiles

1.2 Software Architecture

Layer	Role	Classes / Methods
Optical Engine	Physical calculations (focal length, DoF, FOV, step sizes)	OpticalCalculator
Hardware Profiles	Lens, camera and machine configuration management	LensProfileDialog, CameraProfileDialog, MachineProfileDialog
User Interface	Layout, panels, drag-and-drop, dark theme	CNCApp — _create_* methods

Layer	Role	Classes / Methods
CNC Communication	GRBL protocol over serial port (thread-safe)	send_gcode() + _serial_lock
Sequencer	Multi-axis acquisition loop in a background thread	_sequence_worker()
Logging	Timestamped CSV session logs, file held open throughout	_sequence_worker() — CSV block
3D Visualisation	Lightweight isometric preview engine	_redraw_3d_view()

1.3 Dependencies

- `pyserial` — Serial communication with the GRBL board
- `Pillow (PIL)` — Image handling for hardware profile thumbnails
- Standard library: `tkinter`, `threading`, `json`, `csv`, `math`, `time`, `os`

2. CNC Connection and Manual Control

The "CNC Link & Movement" panel is the main entry point. It manages the physical connection, manual XYZ jogging and movement parameters.

2.1 Serial Connection

Parameter	Value
Protocol	GRBL (standard open-source G-code)
Baud rate	115,200 baud
Connection timeout	1 second
Initialisation sequence	Unlock ($X), absolute coordinates (G90)
Position polling	Every 50 ms via the « ? » status command

On startup the application automatically tries to reconnect to the last saved port. If that fails, the user selects the correct COM port from the dropdown and clicks "Connect CNC".

2.2 Manual XYZ Jogging

The jog buttons (Y+, Y−, X+, X−, Z+, Z−) each spawn a **background daemon thread** that sends the G91 / G0 / G90 sequence without blocking the UI. Step size is controlled by the shared "Step (mm)" field.

> **Button Responsiveness Fix**
>
> Previously, manual_move() blocked the main thread for the full duration of 3 serial exchanges (up to 6 s). Moves are now delegated to _manual_move_worker() via threading.Thread, making buttons respond instantly.

2.3 Movement Parameters

Field	Description
Speed (mm/min)	Feed rate used during the automated acquisition sequence (G1 F...)
Step (mm)	Jog step size shared by all six XYZ directional buttons

2.4 Thread-Safe Serial Communication

All serial writes and reads go through `send_gcode()`, which is protected by a **threading.Lock** (`self._serial_lock`). Only one thread can access the serial bus at a time, preventing byte-interleaving between the position polling thread, move workers and the sequence thread.

2.5 Real-Time Position Display

Coordinates are refreshed every 50 ms by an asynchronous `?` query running in a background thread. Polling is suspended during the acquisition sequence to avoid bus conflicts and resumes automatically when the sequence ends.

3. Hardware Profiles

The "Hardware Setup" panel centralises the configuration for the three physical devices. Each profile type has an Edit button and a ✕ delete button (with a confirmation dialog).

3.1 Lens Profile

Parameter	Description	Example
Nominal Mag	Nominal magnification engraved on the objective	10
Numerical Aperture (NA)	Numerical aperture — determines resolution and depth of field	0.25
Objective Tube (mm)	Nominal tube length (RMS standard = 160 mm)	160.0
Current Physical Length	Actual physical tube length at rest (M = 0)	160.0
Invert X Axis Comp.	Reverses the X compensation direction on M moves	false
Image	Objective photo stored as Base64 JPEG thumbnail (150×150 px)	Optional

3.2 Camera Profile

Parameter	Description	Example
Sensor Width (mm)	Physical width of the image sensor	36.0 (Full Frame)
Sensor Height (mm)	Physical height of the image sensor	24.0
Circle of Confusion (CoC)	Maximum acceptable blur circle — affects effective depth of field	0.03 mm

3.3 Machine Profile

Parameter	Description
Travel X/Y/Z/M (mm)	Maximum travel per axis — used as the M axis progress bar upper bound
Precision X/Y/Z/M (mm)	Minimum axis resolution — computed step sizes are quantised down to this multiple

3.4 Deleting a Profile

Each profile row has a ✕ button next to Edit. Clicking it shows a confirmation dialog before permanent deletion. If the last profile of a given type is deleted, `_update_hw_combos()` automatically recreates a default profile — no invalid application state is possible.

3.5 Persistence

All profiles are stored in `config.json` in the same directory as the executable or script. Images are Base64-encoded JPEG thumbnails. Multiple profiles of each type can coexist and are selected via dropdown lists.

4. Optical Calculation Engine

The `OpticalCalculator` class implements the physical equations used to automatically compute sequence parameters from hardware profiles and the current M axis position.

4.1 Focal Length

```
f = L_tube / (G_nominal + 1)
```

4.2 Effective Magnification and Total Extension

```
Total extension  i = L_rest + M_position
M_effective = (i - f) / f
```

L_rest = physical tube length at rest (lens profile), M_position = current A-axis reading

4.3 Field of View (FOV)

- `FOV_Y = Sensor_Width / M_effective`
- `FOV_Z = Sensor_Height / M_effective`

4.4 Depth of Field (DoF)

$$DoF = \lambda / NA^2 + CoC / (M_effective \times NA)$$

λ = 0.00055 mm (550 nm green light reference), NA = numerical aperture, CoC = circle of confusion

4.5 Step Sizes

- `Step_X = DoF × (1 - Focus_Overlap)` — focus depth stepping axis
- `Step_Y = FOV_Y × (1 - Lat_Overlap)` — lateral axis 1

- `Step_Z = FOV_Z × (1 − Lat_Overlap)` — lateral axis 2

Each step is then quantised down to the nearest multiple of the machine precision for that axis.

4.6 M Axis Optical Compensation

Moving the M axis changes the working distance. An X compensation move is calculated and sent simultaneously to keep the subject in focus:

Working Distance Formula

$WD = (i \times f) / (i - f)$ where i = current total extension, f = focal length

5. Acquisition Sequence

5.1 Defining the Scan Volume

Parameter	Description
Start / End Position (X,Y,Z)	Volume bounds. "Set Start" / "Set End" capture the current machine position
Step (X,Y,Z) mm	Inter-position spacing — entered manually or computed automatically
Lat. Overlap (%)	Lateral overlap between adjacent Y and Z acquisitions (recommended 30–50 %)
Focus Overlap (%)	Axial overlap along X — depth of field overlap (recommended 15–30 %)
Delay (s)	Stabilisation pause — split equally before and after the camera trigger

5.2 Auto-Calculate and Time Estimation

The **"□ Auto-Calculate"** button computes optimal step sizes and symmetrically expands the bounds to fit an exact integer number of steps. The **"Estimate Time & Info"** button calculates the total photo count and estimated duration without starting the sequence.

5.3 Pre-Launch Validation

Before generating scan points, `_sequence_worker` checks that the span/step ratio does not exceed 9,999 for each axis. If the limit is reached, an **orange warning** is appended to the sequence log with the axis name, step size and span. The sequence still launches, but the user is alerted that a step may be too small.

5.4 Scan Algorithm

Level	Axis	Description
Outer loop	Z	Depth layers — successive focus positions
Middle loop	Y	Rows of the lateral mosaic
Inner loop	X	Columns — optical depth axis

At each position the sequence performs:

1. **Move** — G1 to target coordinates at the configured feed rate
2. **Wait for completion** — GRBL polling until Idle state, 45 s timeout
3. **Stabilisation** — half the configured delay before and after the trigger
4. **Camera trigger** — M64 P0 (ON) then M65 P0 (OFF) with a 100 ms hold
5. **CSV logging** — immediate row write + flush to disk
6. **UI update** — progress bar, ETA counter and 3D view refresh

5.5 GRBL Alarm Detection

On every iteration of `_wait_for_move_completion`, the `?` response is tested for the string "alarm" (case-insensitive). If detected: an error message with a reminder to run `$X` is appended to the log, `stop_sequence_flag` is set, and the function returns immediately — no 45 s wait.

> **Unlocking after an alarm**
>
> If a GRBL alarm fires (limit switch hit, etc.), type $X in the GRBL Console panel to unlock the machine before resuming.

5.6 Sequence Controls

Button	Action
Pause	Halts after the current photo (threading.Event)
Resume	Continues from the next position
Stop	Full stop — waits for the current move to finish before exiting

Button	Action
Skip Y Line	Skips remaining X positions on the current Y row and moves to the next

6. CSV Session Logging

The "SD-Card Sync" module records every acquisition position in a timestamped CSV file. The file is opened once at sequence start and held open until the sequence ends.

6.1 Write Strategy

The CSV file is opened with `open()` during log initialisation. A persistent `csv.writer` object is used for the entire sequence. After every data row, `log_file.flush()` forces an OS-level write without closing the file. At sequence end the summary footer is written and the file is closed inside a `finally` block — guaranteed even on exception.

> **Improvement over the previous approach**
>
> The previous implementation opened and closed the CSV file on every single photo (open/close inside the inner loop). With a persistent file handle + flush(), system overhead is negligible while providing the same crash resilience: every photo is immediately persisted to disk.

6.2 File Structure

Metadata Header

Row	Content
# METADATA:	Session start date and time
# LENS:	Name, NA, nominal magnification, computed actual magnification (ActualMag=Xx)
# CAMERA:	Name and sensor dimensions
# PARAMS:	Lateral overlap, focus overlap, delay
# TOTAL CALCULATED PHOTOS:	Planned photo count

Data Columns

Column	Type	Description
frame_index	Integer	Sequential photo number (0-indexed)
timestamp	Float (Unix)	Unix timestamp at the moment of triggering
x_um	Float	X coordinate in µm (mm × 1000)
y_um	Float	Y coordinate in µm
z_um	Float	Z coordinate in µm
magnification	Float	Effective magnification computed from M position at sequence launch

The magnification is computed **once** at sequence start (the M axis does not move during acquisition) and repeated on every row to make the file self-contained for downstream processing tools.

Summary Footer

Row	Content
# SUMMARY:	Session end date and time
# ACTUAL PHOTOS TAKEN:	Actual number of triggers fired
# TOTAL DURATION:	Total elapsed time in seconds

7. Magnification Panel (M Axis)

The "Magnification" panel controls the 4th axis (A in GRBL) which drives the optical tube extension of the microscope objective. It centralises all real-time optical readouts.

7.1 Real-Time Displays

Element	Value displayed	Update rate
M Position	Current A-axis position in mm	Every poll (50 ms)
Total Extension	start_tube_length + M_position = actual image distance i (mm)	Every poll
Progress bar	M travel used vs. machine profile maximum travel	Every poll
Current Magnification	M_eff = (i − f) / f computed from total extension	Every poll

The **Total Extension** is the actual optical path length between the objective and the sensor. It equals the value `i` used in all optical formulas, allowing direct cross-checking against objective manufacturer specifications.

7.2 M Step

The **"M Step (mm)"** field in this panel is **independent** from the XYZ Step in the CNC panel. It exclusively controls the amplitude of the M− and M+ buttons. This separation allows fine-grained steps for optical adjustment on the M axis while keeping a coarser step for XYZ stage positioning.

7.3 M Moves with Optical Compensation

Every M−/M+ press spawns a background thread (_move_m_worker). If a valid lens profile is active, the M move is paired with a simultaneous X compensation to keep the subject in focus:

7. `G91` — switch to relative positioning mode

8. `G0 A{step_m} X{dx_compensation}` — simultaneous M + X move in one command
9. `G90` — return to absolute positioning mode

Without an active lens profile only the A axis moves. The compensation direction can be reversed via the "Invert X" flag in the lens profile.

8. 3D Isometric Preview

The "3D Sequence View" panel renders an isometric view of the scan volume. The engine is intentionally lightweight (pure Tkinter Canvas, no 3D library) to run smoothly on modest hardware.

8.1 Visual Elements

Element	Rendering	Colour
Bounding box — hidden edges	Dashed thin lines (drawn first, behind everything)	Sky blue
Progress cubes Z/Y/X	Solid faces with no outline	Dark blue gradient
Current FOV frame	Hatched rectangle representing the camera field of view	Cyan accent
Bounding box — visible edges	Solid lines (drawn last, always on top)	Sky blue
Start label	First actual scan point (x_pts[0], y_pts[0], z_pts[0])	Green
End label	Last actual scan point (x_pts[-1], ...)	Red

8.2 Draw Order (Painter's Algorithm)

The render follows a strict order to prevent visual artefacts from overlapping geometry:

10. Hidden bounding box edges (dashed) — behind all other elements
11. Progress cube faces (solid fill, no outline)
12. Current FOV frame
13. Visible bounding box edges — drawn last so they always read clearly over the filled cubes
14. Start / End labels

Progress cubes have **no outline** (`outline=""`). The previous version used a bright blue outline that overlapped the bounding box edges and created confusing optical illusions.

8.3 Corrected Start / End Labels

Labels now point to the **actual first and last scan points** (`x_pts[0], y_pts[0], z_pts[0]` and `x_pts[-1]...`). Previously they were fixed to the min/max corners of the bounding box, which was wrong when a scan ran in a decreasing direction.

8.4 Isometric Projection

- `u = (ny − nx) × cos(30°)` — horizontal screen component
- `v = −nz + (nx + ny) × sin(30°)` — vertical screen component

nx, ny, nz: coordinates normalised relative to the volume centre

The view auto-scales to the panel size and updates in real time at every acquired position during the sequence.

9. GRBL Console

The "GRBL Console" panel allows sending raw G-code commands directly to the machine and reading its responses.

Command	Action
?	Request current state and position
$$	List all machine configuration parameters
$X	Unlock machine after an alarm condition
$H	Homing cycle — return to limit switches
G92 X0 Y0 Z0	Set work coordinate origin at current position
!	Feed Hold — emergency stop of the current motion

Sent commands are displayed in sky blue, received responses in green. History is capped at 500 lines. The console is thread-safe through the `_serial_lock` shared with all other workers.

10. User Interface and Layout

10.1 Panel Organisation

Default column	Panels
Left	CNC Link & Movement, Camera Control, SD-Card Sync, Magnification
Centre	Hardware Setup, Focus Stacking Sequence, 3D Sequence View
Right	GRBL Console, Sequence Log

10.2 Drag & Drop Layout

Click-hold on a panel title bar (top 25 pixels) to start dragging. A ghost label follows the cursor and a blue horizontal indicator shows the insertion target. The panel arrangement and pane divider positions are saved automatically on close and fully restored on next launch.

10.3 Dark Theme

Role	Hex	Used for
Primary background	#0A0E17	Root window, text areas, 3D canvas
Panel background	#111827	LabelFrames, containers
Widget background	#1F2937	Entries, comboboxes, image preview areas
Primary accent	#38BDF8	Panel titles, coordinate readouts, GRBL console sent commands
Success green	#22C55E	GRBL response text, 3D Start label
Error red	#EF4444	Error messages, 3D End label
Warning amber	#F59E0B	Sequence log warnings

11. Typical Workflow

Step 1 — Connect

15. Plug the CNC controller in via USB
16. Launch the software — automatic reconnection to the last saved port
17. If it fails: select the correct COM port and click "Connect CNC"

Step 2 — Hardware Setup

18. Select or create a Lens Profile (magnification, NA, tube length)
19. Select or create a Camera Profile (sensor dimensions, CoC)
20. Select the Machine Profile

Step 3 — Positioning

21. Use the XYZ jog buttons to position the stage over the subject
22. Set M Step then use M−/M+ to dial in the desired magnification
23. Monitor M Position, Total Extension and Current Magnification in real time
24. Move to the scan start point → **Set Start**; move to the end point → **Set End**

Step 4 — Sequence Setup

25. Configure the lateral and focus overlap percentages
26. **Auto-Calculate** → review computed step sizes and total photo count
27. **Estimate Time** → validate the estimated duration
28. Check the 3D preview for volume consistency

Step 5 — Acquisition

29. Enable CSV logging and verify the output folder path
30. **Start Sequence**
31. Monitor progress — use Pause / Skip / Stop as needed
32. An audio cue plays on normal sequence completion

Step 6 — Data Retrieval

33. "Open Logs Folder" → access the session CSV file
34. Import into Helicon Focus, Zerene Stacker, or equivalent stacking software
35. Use x_um, y_um, z_um and magnification columns for downstream processing

12. Complete Parameter Reference

12.1 Sequence Parameters

Parameter	Unit	Default	Description
Start / End X,Y,Z	mm	0.0 / 10.0	Scan volume bounds
Step X,Y,Z	mm	1.0	Inter-position spacing (manual or auto-computed)
Lat. Overlap	%	40	Lateral overlap for Y and Z axes
Focus Overlap	%	20	Axial overlap along X (depth of field)
Delay	s	0.5	Stabilisation pause — split before and after trigger
Speed	mm/min	1000	Feed rate for sequence moves
Step (XYZ jog)	mm	1	Manual jog step size — CNC Link & Movement panel
M Step	mm	1	M axis step — Magnification panel, independent from XYZ

12.2 Connection Parameters

Parameter	Value
Baud rate	115,200 baud (fixed)
Read / write timeout	1 s / 2 s
G-code response timeout	2.0 s (0.25 s for status « ? »)
Move completion timeout	45 s
Position poll interval	50 ms (suspended during sequence)

12.3 Optical and System Constants

Constant	Value	Description
Wavelength λ	0.00055 mm (550 nm)	Green light reference for diffraction-limited DoF
Camera trigger ON/OFF	M64 P0 / M65 P0	GRBL I/O commands for shutter control
Trigger hold duration	100 ms	Minimum shutter signal duration
Scan point limit	10,000 / axis	Safety cap with warning logged if approached
Console history limit	500 lines	GRBL console and sequence log

12.4 config.json Key Reference

- `com_port` — last used COM port
- `start_pos / end_pos / step_seq` — sequence positions and step sizes
- `cnc_step` — single XYZ jog step size
- `step_m` — independent M axis step size
- `lens_profiles / camera_profiles / machine_profiles` — profile dictionaries including Base64 images
- `active_lens / active_camera / active_machine` — currently selected profiles
- `layout / sashes` — panel arrangement and divider positions
- `log_enabled / log_dir` — CSV logging on/off and output folder

13. Complementary Software Suite

CMCS (CNC Microphotography Control Software, also referred to as MicroPhotoCNC) is part of a larger software ecosystem designed to cover the full acquisition-to-stacking pipeline. Three complementary tools are provided alongside the main control software:

Post-Watcher (PW) — ingestion, sorting and preprocessing pipeline

Dust Map Profiler — flat-field calibration for vignetting and dust correction

Poisson Profiler — photon shot-noise calibration for Anscombe denoising

All tools output or consume a shared sidecar JSON format that encodes per-frame coordinates, timestamps, optical metadata and preprocessing parameters. This format is designed to serve as the input layer for a future custom stacking engine — the JSON files carry all the information a stacking algorithm needs to register and blend frames without manual intervention.

13.1 Post-Watcher (PW)

13.1.1 Purpose

Post-Watcher is a desktop tool (Python 3 / Tkinter) that automates the entire post-acquisition workflow: it ingests the raw image files from the camera's SD card, cross-references them against the CNC session CSV log, sorts the frames into per-stack folders, writes JSON sidecar metadata, and optionally runs a full preprocessing pipeline before export.

The tool supports both RAW files (`.rw2, .cr3, .nef, .dng, .arw`) and standard image formats (`.jpg, .jpeg, .tif, .tiff, .png`). RAW decoding is handled via `rawpy` with linear gamma and camera white balance preserved — all preprocessing operates on linear 16-bit data for photometric accuracy.

13.1.2 Workflow Phases

Phase 1 — Ingestion & Sorting

Parse the CNC session CSV log to extract the ordered list of acquisition positions (frame_index, x_um, y_um, z_um, timestamp).

Scan the SD card folder recursively, sorting all image files by modification time.

Match the N most recent files with the N CSV entries (older photos from previous sessions are automatically ignored).

Create an export directory with timestamped naming (e.g. 250508_1730_450pics_15x30).

For each frame, copy the original file into a per-stack subfolder named by its Y/Z grid position (e.g. Y1250.0_Z0.0/) and write a sidecar JSON file containing frame_index, x_um, y_um, z_um, original_timestamp (Unix) and metadata_context (full CSV header metadata).

Provide a temporal coherence check between the CSV timestamps and SD card file times — a >25 s discrepancy triggers a visual warning.

Phase 2 — Preprocessing (optional)

When enabled, Post-Watcher processes each stack folder sequentially through a configurable pipeline of three independent steps:

Step	Description	Dependencies	Output
Flat-Field Correction	Divides each frame by a normalized Dust Map to remove vignetting, sensor dust shadows and uneven illumination. The Dust Map is loaded from a .tif file generated by the Dust Map Profiler. Separate profiles can be specified for RAW and JPG inputs — the tool auto-detects the dominant image type.	Dust Map .tif (tifffile or PIL)	Uniformly illuminated frames with dust artifacts removed
ORB Alignment	Computes pairwise affine transformations between consecutive frames using ORB features (OpenCV). Frames below a Laplacian variance threshold (50.0) are skipped to avoid aligning blurry frames. A cumulative transform chain is built relative to the middle frame of each stack.	OpenCV (cv2) NumPy	Registered (aligned) frames
Anscombe Denoising	Applies a variance-stabilising Anscombe transform followed by Gaussian filtering, then inverse transform. Uses the photon_scale parameter from the Poisson Profiler to correctly model shot-noise variance. Only effective when photon_scale is accurately calibrated.	photon_scale from Poisson Profiler (JSON profile)	Denoised frames with preserved detail

13.1.3 Output Format Selection

Processed frames are exported as TIFF files in one of two formats:

`float32 linear` — 32-bit floating point, linear RGB. Ideal for machine learning pipelines, custom stacking algorithms, and any workflow requiring photometric linearity.

`uint16 gamma sRGB` — 16-bit integer with 2.2 gamma correction. Compatible with commercial stacking software such as Helicon Focus and Zerene Stacker.

13.1.4 Safety Features

Source file preservation — original files can be copied (archive mode) or processed in-place. A delete-after-export option is available with a confirmation dialog.

Processing error resilience — if any stack fails during Phase 2, the error is logged and processing continues with the remaining stacks. Source deletion is automatically blocked if any preprocessing error occurred.

Disk-full detection — an ENOSPC (no space left) error triggers a clear message asking the user to free space and relaunch.

Session persistence — all UI settings are saved in post_watcher_settings.json and restored on next launch.

13.2 Dust Map Profiler

13.2.1 Purpose

The Dust Map Profiler (Phase 0.4 of the calibration pipeline) builds a flat-field reference map from a set of uniform illumination frames (flat frames). The resulting Dust Map encodes the combined vignetting pattern and dust shadow map of the full optical train — from the objective lens to the camera sensor.

13.2.2 Algorithm

Load a folder of flat frames — shot against a uniformly illuminated surface at the same ISO as production images (typically ISO 100). A minimum of 3 frames is required; 5–10 is recommended.

Decode each frame to linear 16-bit float32 using rawpy (RAW) or PIL (JPEG) — no gamma, no auto-brightness, camera white balance preserved.

Compute the pixel-wise median across all frames to eliminate temporal noise:

$$\text{DustMap}(x, y, c) = \text{median}(\text{FlatFrame}_1(x, y, c), \ldots, \text{FlatFrame}_n(x, y, c))$$

Evaluate quality via a 500×500 pixel central region on the green channel:

— Mean luminance (normalised 0–1)

— Coefficient of variation in the central patch (must be < 0.5 % for GO status)

Export as 16-bit TIFF (float32) and/or register in config.json with objective name, ISO, exposure times and calibration date.

13.2.3 Usage in Post-Watcher

The Dust Map produced by this tool is consumed by Post-Watcher's flat-field correction step. During preprocessing, each frame is divided by the normalised Dust Map:

```
corrected(x, y, c) = frame(x, y, c) / (DustMap(x, y, c) / mean(DustMap[:, :, c]))
```

This removes sensor-fixed pattern noise without affecting the overall exposure level. Separate Dust Maps can be specified for RAW and JPEG stacks — Post-Watcher automatically selects the appropriate one based on the dominant image type in the stack.

13.2.4 Design Notes

No dark frame subtraction — at the short exposures used for flat fields (typically 1/60 s at ISO 100), CMOS sensor thermal noise is below the noise floor.

The application includes a built-in validation viewer that displays: the Dust Map in false colour (Inferno colormap), the original production frame, and the corrected result.

The green channel is used for quality metrics because it carries the most luminance information in Bayer-filtered sensors.

13.3 Poisson Profiler

13.3.1 Purpose

The Poisson Profiler (Phase 0.5 of the calibration pipeline) measures the photon shot-noise characteristics of the camera sensor to compute a photon_scale parameter. This value is required by the Anscombe variance-stabilising transform used in Post-Watcher's denoising step.

13.3.2 Theory

Photon arrival at the sensor follows Poisson statistics. For a linear sensor response:

Variance = Mean / photon_scale

where photon_scale (in electrons per digital unit) encodes the sensor gain. By measuring the mean intensity and pixel-wise variance across multiple uniform flat frames, we can estimate:

photon_scale = mean(intensity) / variance

A higher photon_scale indicates lower noise (more photons per digital count). This parameter is essential for correct noise modelling in the Anscombe transform — without it the denoising filter would either over-smooth or leave excessive noise.

13.3.3 Algorithm

Load a folder of flat frames shot at ISO 100 (shot noise must dominate over read noise).

Decode each frame to linear float32 (same pipeline as Dust Map Profiler).

Extract the central 500×500 pixel patch from the green channel of each frame.

Compute the global mean intensity across all patches.

Compute the per-pixel variance across frames, then take the spatial mean of that variance.

photon_scale = mean / variance

Constraints: minimum 3 frames (5–10 recommended), uniform exposure, identical ISO. The ISO must match the production images that will be denoised later.

13.3.4 Visualization

Noise distribution histogram — shows the distribution of all green-channel pixels across all frames, with the mean value marked and the photon_scale displayed.

Spatial variance map — false-colour rendering (Inferno colormap) of the per-pixel variance, revealing spatial patterns in sensor noise (e.g. higher variance in brighter regions).

13.3.5 Output

The calibration result is saved as a JSON entry in `config.json`:

```
"poisson_calibration": {
  "photon_scale": 245.3,
  "mean": 0.123456,
  "variance": 0.000503,
  "n_frames": 8,
  "date": "2025-07-15"
}
```

Post-Watcher reads this value to parameterise the Anscombe denoising step. As with the Dust Map, separate profiles can be maintained for RAW and JPEG inputs.

13.4 Calibration Pipeline Overview

The three complementary tools form a coherent pre-acquisition and post-acquisition pipeline:

Phase	Tool	When	What it produces	Consumed by
0.4	Dust Map Profiler	Before session (once per objective/ISO)	Dust Map .tif config.json entry	Post-Watcher (flat-field correction)
0.5	Poisson Profiler	Before session (once per camera/ISO)	photon_scale in config.json	Post-Watcher (Anscombe denoising)
1	CMCS (MicroPhotoCNC)	During session	Image files on SD card CSV session log	Post-Watcher
2	Post-Watcher	After session	Sorted folders Sidecar JSON Preprocessed TIFFs	Stacking software (Future custom stacker)

The Dust Map Profiler and Poisson Profiler are calibration tools that need to be run once per hardware configuration (objective + ISO combination). Their output is stored persistently in config.json and automatically referenced by Post-Watcher during preprocessing. This separation of calibration from processing ensures reproducibility — the same calibration data can be applied to multiple acquisition sessions.

13.5 Sidecar JSON Format Specification

13.5.1 Rationale

Each image frame in the export tree is accompanied by a sidecar JSON file with the same base name (e.g. `img_0042_X15200.0.json` next to `img_0042_X15200.0.rw2`). This format serves two immediate purposes — providing metadata to existing stacking software and enabling automated folder-based batch processing — but its primary design goal is to become the standard input format for a future custom stacking engine.

The sidecar encodes everything a stacking algorithm needs to know about a frame without requiring access to the original CSV log or acquisition software:

13.5.2 Schema

Field	Type	Description

frame_index	integer	0-based sequential photo number from the acquisition sequence
x_um	float	X coordinate in micrometres (focus/depth axis), mm × 1000
y_um	float	Y coordinate in micrometres (lateral axis 1)
z_um	float	Z coordinate in micrometres (lateral axis 2)
original_timestamp	float (Unix)	Unix timestamp at the moment of camera trigger
metadata_context	object	Full metadata header from the CSV session log including lens profile, camera profile, overlap percentages, total photo count, and session timestamps

13.5.3 Example

```
{
  "frame_index": 42,
  "x_um": 15200.0,
  "y_um": 1250.0,
  "z_um": 0.0,
  "original_timestamp": 1751554321.5,
  "metadata_context": {
    "METADATA": "2025-07-03 14:25:01",
    "LENS": "Nikon M Plan 10x — NA=0.25 — ActualMag=10.2x",
    "CAMERA": "Sony A7R3 — 35.9x24.0mm",
    "PARAMS": "LatOverlap=40% | FocusOverlap=20% | Delay=0.5s",
    "TOTAL CALCULATED PHOTOS": "450"
  }
}
```

13.5.4 Export Tree Structure

The Post-Watcher export directory follows this layout:

```
250508_1730_450pics_15x30/            ← timestamped session folder
├── Y0.0_Z0.0/                         ← stack folder (Y/Z grid position)
│   ├── img_0000_X10000.0.rw2          ← original (if copy enabled)
│   ├── img_0000_X10000.0.json         ← sidecar metadata
│   ├── img_0001_X10050.0.rw2
│   ├── img_0001_X10050.0.json
```

```
│   ├── ...
│   └── processed/                    ← preprocessed TIFFs (if Phase 2 enabled)
│       ├── img_0000_X10000.0.tif
│       ├── img_0001_X10050.0.tif
│       └── ...
├── Y1250.0_Z0.0/
│   └── ...
└── Y2500.0_Z0.0/
    └── ...
```

Each Y/Z subfolder contains one focus stack (all X positions for that lateral grid cell). The processed/ subdirectory holds the preprocessed output in TIFF format, ready for direct import into any stacking software.

13.5.5 Future Stacking Engine Integration

Coordinates in micrometres (not pixels) — all spatial information is expressed in physical units, making it independent of sensor resolution and magnification. A stacker can compute pixel-space displacements from the sensor dimensions and magnification stored in metadata_context.

Self-contained metadata — each sidecar carries the full acquisition context. A stacking engine can reconstruct the entire scan geometry (grid layout, overlap percentages, optical parameters) from a single folder of JSON files without any external configuration.

Format stability — the JSON schema uses simple types (integer, float, object) with no dependencies on external libraries or binary formats. It is trivial to parse in any language (Python, Rust, C++, JavaScript).

Preprocessing provenance — when Phase 2 is enabled, the exported TIFF files in processed/ inherit the same base name as the sidecar JSON, making it straightforward to associate preprocessed data with its original acquisition parameters.

14. Complete Software Dependencies

Package	Version	Used by	Purpose
Python 3	≥ 3.10	All tools	Runtime environment
tkinter	stdlib	All tools	GUI framework
pyserial	≥ 3.5	CMCS	Serial communication with GRBL
Pillow (PIL)	≥ 9.0	All tools	Image loading, display, thumbnail encoding
rawpy	≥ 0.18	PW, DustMap, Poisson	RAW file decoding with LibRaw
opencv-python (cv2)	≥ 4.8	PW, DustMap, Poisson	Image processing, ORB alignment, colormaps
numpy	≥ 1.24	PW, DustMap, Poisson	Numerical arrays and operations
tifffile	≥ 2024	PW	Float32 TIFF read/write (recommended)